COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 007 Programming & Design	**SECTION:**	CPE11s4
DATE PERFORMED:	10 -28-26	**DATE SUBMITTED:**	10-28-26
NAME:	Sean Andrei P Evangelista	**INSTRUCTOR:**	Robin Valenzuela

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Task 1 (library borrowing system)

Write a program that manages a library catalog stored in books.txt. Define struct Book { int id; string title; int copies; }; where each title is one word. Create a function loadBooks() that reads every record of books.txt into an array of Book structures and returns the count; if books.txt cannot be opened, it must first call createCatalog(), which writes the records 101 Algorithms 3, 102 Networks 0, 103 Databases 2, and 104 Electronics 1. In main(), use a loop that asks for a book ID until the user enters 0. A function borrowBook() searches the array. If the ID is not found, it should print the message: "Error, no such book." If no copies are left, it should print: "Sorry, no copies left." Otherwise, it lowers the copies by one and prints "Borrowed: " followed by the title and the copies left. After each successful borrow, saveBooks() must rewrite books.txt and logTransaction() must add the line BORROW and the ID to log.txt; choose the correct stream and mode for each. At the end, print the number of successful transactions. Your version of the program must print the same result as the expected output (assume books.txt doe

```cpp
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

    struct Book {
        int id;
        string title;
        int copies;
};
void createCatalog(){
    ofstream file("books.txt");
    file << "101 Algorithms 3" << endl;
    file << "102 Networks 0" << endl;
    file << "103 Databases 2" << endl;
    file << "104 Electronics 1" << endl;
    file.close();
}
    int loadBooks(Book books[]){
        ifstream file("books.txt");
        if (!file.is_open()){
        createCatalog();
        file.open("books.txt");
    }

    int count = 0;
    while (file >> books[count].id >> books[count].title >> books[count].copies){
        count++;
    }

    file.close();
    return count;
}
void saveBooks(Book books[], int count){
    ofstream file("books.txt");
    for (int i = 0; i < count; i++){
        file << books[i].id << " "
        << books[i].title << " "
        << books[i].copies << endl;
    }
    file.close();
}
void logTransaction(int id){
    ofstream file("log.txt", ios::app);
    file << "BORROW " << id << endl;
    file.close();
}
```

s not exist yet)

```cpp
bool borrowBook(Book books[], int count, int id){
    for (int i = 0; i < count; i++){
        if (books[i].id == id){
            if (books[i].copies == 0){
                cout << "Sorry, no copies left." << endl;
                return false;
            }
            books[i].copies--;
            cout << "borrowed: " << books[i].title << " ("
            << books[i].copies << " left)" << endl;
            saveBooks( books, count );
            logTransaction( id );
            return true;
        }
    }
    cout << "Error, no such book." << endl;
    return false;
}
int main(){
    Book books[100];
    int id;
    int transactions = 0;
    int count = loadBooks(books);
    while (true){
        cout << "Enter book ID (0 to stop): ";
        cin >> id;
        if (id == 0){
            break;
        }
        if (borrowBook(books, count, id)){
            transactions++;
        }
    }
    cout << "Transactions saved: " << transactions << endl;
    return 0;
}
```

Task 2 (payroll report):
Write a program that generates a payroll report with overtime pay. Define struct Employee { string name; double rate; int hours; }; and create a function createFile() that writes these five records to employees.txt, one per line in the format name rate hours: Ana 150 40, Ben 120 45, Carla 200 38, Dan 130 50, and Ella 180 42. Create a function computePay(Employee e) that returns the employee's pay: hours up to 40 are paid at the normal rate, and every hour beyond 40 is paid at 1.5 times the rate. Create a function writePayroll() that reads employees.txt record by record into an Employee variable and writes the heading PAYROLL REPORT and one line per employee (name and pay, plus OT if the employee worked more than 40 hours) to payroll.txt, while tracking the total payroll, the highest-paid employee, and the number of overtime employees. Create a function appendSummary() that adds these three results to payroll.txt in append mode, and a function displayFile(string filename) that prints payroll.txt line by line using getline(). Every function must check that its file opened and close it after use. Your version of the program must print the same result as the expected output.

```cpp
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

struct Employee {
    string name;
    double rate;
    int hours;
};

void createFile() {
    ofstream out("employee.txt");
    if (!out) return;
    out << "Ana 150 40" << endl;
    out << "Ben 120 45" << endl;
    out << "Carla 200 38" << endl;
    out << "Dan 130 50" << endl;
    out << "Ella 180 42" << endl;
    out.close();
}


double computePay(Employee worker) {
    if (worker.hours <= 40) {
        return worker.hours * worker.rate;
    } else {
        return (40 * worker.rate) + ((worker.hours - 40) * worker.rate * 1.5);
    }
}

void writePayroll() {
    ifstream in("employee.txt");
    if (!in) return;

    ofstream out("payroll.txt");
    if (!out) {
        in.close();
        return;
    }

    Employee worker;
    double total = 0;
    double highestPay = -1.0;
    string topWorker = "";
    int otCount = 0;
```

```
    while (in >> worker.name >> worker.rate >> worker.hours) {
        double pay = computePay(worker);
        total += pay;

        if (pay > highestPay) {
            highestPay = pay;
            topWorker = worker.name;
        }

        out << worker.name << " " << pay;
        if (worker.hours > 40) {
            otCount++;
            out << " OT";
        }
        out << "\n";
    }

    in.close();
    out.close();

    ofstream app("payroll.txt", ios::app);
    if (!app) return;
    app << "Total Payroll: " << total << "\n";
    app << "Highest Paid: " << topWorker << "\n";
    app << "Overtime Employees: " << otCount << "\n";
    app.close();
}

void displayFile(string filename) {
    ifstream in(filename);
    if (!in) return;
    string line;
    while (getline(in, line)) {
        cout << line << endl;
    }
    in.close();
}

int main() {
    createFile();
    writePayroll();
    displayFile("payroll.txt");
    return 0;
}
```

```
PAYROLL REPORT
Ana 6000
Ben 5700 OT
Carla 7600
Dan 7150 OT
Ella 7740 OT
Total payroll: 34190
Highest paid: Ella 7740
Overtime employees: 3

```

```
2001 Ana 86
2002 Ben 78
2003 Carla 91

```

```
Ana 150 40
Ben 120 45
Carla 200 38
Dan 130 50
Ella 180 42

```

Task 3 (student records manager):
Write a menu-driven program that maintains student records in students.txt. Define struct Student { int id; string name; double grade; }; and create a function loadStudents() that reads every record into an array of Student structures and returns the count; if students.txt cannot be opened, it must first create the file with 2001 Ana 86, 2002 Ben 72, 2003 Carla 91, and 2004 Dan 68. Use a loop that shows the menu 1) Update 2) Drop 3) Show all 4) Save and quit until the user chooses 4. Option 1 reads an ID and a new grade: updateGrade() prints "Error, no such student." if the ID is not found, "Error, invalid grade." if the grade is not from 0 to 100, and otherwise changes the grade and prints "Updated: " followed by the name and the new grade. Option 2 reads an ID: dropStudent() removes the record from the array by shifting the remaining records to the left and prints "Dropped: " followed by the name, or "Error, no such student." Option 3 prints every record and the class average. Each successful update or drop must be added to changes.txt as UPDATE or DROP followed by the ID. Option 4 rewrites students.txt with the remaining records and prints the number of records saved. Design the program with separate functions, check every file you open, and close it after use. Your version of the program must print the same result as the expected output (assume students.txt does not exist yet)

```cpp
#include <fstream>
#include <string>
#include <iostream>
using namespace std;

struct Student {
    int id;
    string name;
    double grade;
};

void createStudentFile() {
    ofstream out("student.txt");
    if (!out) return;
    out << "2001 Ana 86" << endl;
    out << "2002 Ben 72" << endl;
    out << "2003 Carla 91" << endl;
    out << "2004 Dan 68" << endl;
    out.close();
}

int loadStudent(Student records[]) {
    ifstream in("students.txt");
    if (!in) { createStudentFile(); in.open("student.txt"); }
    int items = 0;
    while (in >> records[items].id >> records[items].name >> records[items].grade) items++;
    in.close();
    return items;
}

void logChange(string type, int id) {
    ofstream out("change.txt", ios::app);
    if (out) {
    out << type << " " << id << "\n";
    out.close();
    }
}

void updateGrade(Student records[], int total) {
    int id; double mark;
    cin >> id >> mark;
    for (int i = 0; i < total; i++) {
        if (records[i].id == id) {
            if (mark < 0 || mark > 100) {
                cout << "Error\n";
            return;
        }
```

```
48                records[i].grade = mark;
49                cout << "Updated: " << records[i].name << " " << mark << "\n";
50                logChange("UPDATE", id);
51                return;
52            }
53        }
54        cout << "Error\n";
55    }
56
57    int main() {
58
59
60
61
62
63        return 0;
64    }
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.